Universidade Federal do Ceará - Campus Quixadá
QXD0010 – Estruturas de Dados
Prof. Atílio Gomes Luiz

Avaliação Continuada 07

Leia atentamente as instru¸c˜oes abaixo.

Instru¸c˜oes:

- Esta atividade deve ser feita **individualmente** e deve ser implementado usando a linguagem de programa¸c˜ao C++
- Identifique o seu c´odigo-fonte colocando o seu **nome** e **matr´ıcula** como coment´ario no in´ıcio de seu c´odigo.
- Indente corretamente o seu c´odigo para facilitar o entendimento. **Atividades com c´odigos maus indentados sofrer˜ao redu¸c˜ao na nota.**
- As estruturas de dados devem ser implementadas como TAD. Deve haver um arquivo de cabe¸calho (.h) e um arquivo de implementa¸c˜ao (.cpp) para cada estrutura de dado programada.
- Os programas-fonte devem estar devidamente organizados e documentados. o: Lembre-se de desalocar os endere¸co
- Observa¸c˜as de mem´oria alocados quando os mesmos n˜ao forem mais ser usados.•

DICA: COMECE A ATIVIDADE O QUANTO ANTES.

1 Listas Circulares Duplamente Encadeadas

A estrutura de lista simplesmente encadeada, vista durante a aula, caracteriza-se por formar um encadeamento simples entre os nós: cada nó armazena um ponteiro para o próximo elemento da lista. Dessa forma, não temos como percorrer eficientemente os elementos em ordem inversa. O encadeamento simples também dificulta a retirada de um elemento da lista. Mesmo se tivermos o ponteiro do elemento que desejamos retirar, temos de percorrer a lista, elemento por elemento, para encontrar o elemento anterior, pois, dado o ponteiro para um determinado elemento, não temos como acessar diretamente seu elemento anterior.

Para solucionar esses problemas, podemos formar o que chamamos de **listas duplamente encadeadas**. Nelas, cada elemento tem um ponteiro para o próximo elemento e um ponteiro para o elemento anterior. Assim, dado um elemento, podemos acessar os dois elementos adjacentes: o próximo e o anterior. A lista duplamente encadeada pode ou não ter um nó cabeça e pode ou não ser circular, conforme as conveniências do programador. Uma **lista circular duplamente encadeada com Nó Cabeça** é uma lista duplamente encadeada na qual o último elemento da lista passa a ter como próximo o primeiro elemento, que, por sua vez, passa a ter o último como anterior. A Figura 1 ilustra uma lista duplamente encadeada com estrutura circular e a presença de um nó cabeça.

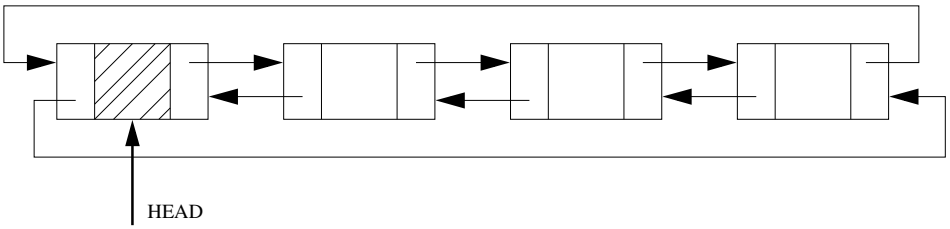


Figura 1: Lista circular duplamente encadeada com nó cabeça.

Problema: Implemente em C++ o Tipo Abstrato de Dados Lista Linear usando como base a estrutura de dados Lista Circular Duplamente Encadeada com Nó Cabeça. A sua estrutura de dados deve ser encapsulada por uma classe chamada `List`, que deve suportar as seguintes operações:

- `List()`: Construtor da classe. Deve iniciar todos os atributos da classe com valores válidos.
- `~List()`: Destrutor da classe. Libera memória previamente alocada.
- `void push_back(int key)`: Insere um inteiro *key* ao final da lista.
- `int pop_back()`: Remove elemento do final da lista e retorna seu valor.
- `void insertAfter(int key, int k)`: Insere um novo nó com valor *key* após o k-ésimo nó da lista.
- `void remove(int key)`: Remove da lista a primeira ocorrência do inteiro *key*
- `void removeAll(int key)`: Remove da lista todas as ocorrências do inteiro *key*.

- `void print()`: Imprime os elementos da lista.
- `void printReverse()`: Imprime os elementos da lista em ordem reversa.
- `bool empty()`: Retorna `true` se a lista estiver vazia e `false` caso contrário.
- `int size()`: Retorna o número de nós da lista.
- `void clear()`: Remove todos os elementos da lista e deixa apenas o nó cabeça.
- `void concat(List& lst)`: Concatena a lista atual com a lista `lst` passada por parâmetro. Após essa operação ser executada, `lst` será uma lista vazia, ou seja, o único nó de `lst` será o nó cabeça.
- `List operator=(const List& lst)`: Retorna uma cópia da lista passada como parâmetro.
- `bool operator==(const List& lst)`: Determina se a lista passada por parâmetro é igual à lista em questão. Duas listas são iguais se elas possuem o mesmo tamanho e o valor do k-ésimo elemento da primeira lista é igual ao k-ésimo elemento da segunda lista.
- `bool operator!=(const List& lst)`: Determina se a lista passada por parâmetro é diferente da lista em questão.

1.1 Interface com o usuário

Escreva um programa principal (`main.cpp`) que interaja com o usuário pelo terminal por meio de comandos, assim como as atividades que eu coloco no Moodle. O usuário deve poder digitar os comandos e testar TODAS as operações da estrutura `List` que você implementou. Os comandos podem ser similares aos comandos utilizados nas atividades do Moodle.

Essa atividade vale de 0 a 10 pontos.

1.2 O que deve ser submetido

- **Deverá ser submetido:**
  - Um pequeno **relatório do trabalho** realizado, contendo uma descrição dos que foram implementados para serem usadas via terminal.
  - Os programas fonte devidamente organizados e documentados.
- Um dos parâmetros utilizados na avaliação da qualidade de uma implementação consiste na constatação da presença ou ausência de comentários. Comente o seu código. Mas também não comente por comentar, forneça bons comentários.
- Não serão aceitas atividades submetidas após o prazo final.